UNIVERSITÉ — FILIÈRE BDCC

Big Data, Cloud Computing et Développement

Rapport de travaux dirigés

Architecture Microservices E-Bank

PARTIE II

Agent bancaire intelligent, Spring AI, MCP, Telegram et Discord

Réalisé par : Abdelali Saadali

Encadré par : Pr. Mohamed YOUSSFI

Formation : BDCC — Semestre 5

Année : 2026–2027

Résumé

Ce rapport documente la deuxième partie du projet académique *E-Bank*, consacrée à l'introduction d'un assistant bancaire fondé sur Spring AI et sur le *Model Context Protocol* (MCP). L'existant comprend deux microservices métier (*Customer-Service* et *EBank-Service*), une découverte de services Eureka et une passerelle Spring Cloud Gateway. La nouvelle application *EBank-Bot* enrichit cette architecture d'un agent conversationnel, d'une mémoire de discussion et de cinq outils MCP en lecture seule, ainsi que de deux interfaces de messagerie, Telegram et Discord. Les captures montrent la découverte des outils et leur exécution dans MCP Inspector, l'enregistrement des applications dans Eureka et la présence des canaux de messagerie. Une tentative Swagger retourne également une erreur HTTP 500 : elle est rapportée comme limite observée et non comme succès du modèle. L'objectif est d'exposer une démarche reproductible tout en distinguant clairement les résultats démontrés des fonctionnalités encore à valider.

Mots-clés : microservices, Spring Boot, Spring AI, MCP, Eureka, Gateway, Telegram, Discord.

Table des matières

Résumé **i**

1 Introduction générale **1**
1.1 Contexte et continuité du projet 1
1.2 Objectifs 1
1.3 Environnement 1

2 Architecture technique de la partie II **2**
2.1 Organisation des composants 2
2.2 Principes de conception 2
2.3 Enregistrement Eureka 2

3 Création du microservice intelligent **5**
3.1 EBank-Bot et les dépendances 5
3.2 Agent conversationnel 5
3.3 API REST et Swagger 5

4 Publication des outils MCP **7**
4.1 Principes du Model Context Protocol 7
4.2 Serveur Customer 7
4.3 Serveur EBank 7
4.4 Déclaration dans MCP Inspector 7
4.5 Exécution des outils 9

5 Intégration du client MCP et de Gateway **12**
5.1 Transport via Gateway 12
5.2 Client MCP Spring AI 12
5.3 Difficultés de découverte 12

6 Intégration des interfaces de messagerie **14**
6.1 Telegram 14
6.2 Discord 14
6.3 Mutualisation et limites 14

7 Bilan des validations et perspectives **17**
7.1 Matrice des preuves 17
7.2 Améliorations envisageables 17

Conclusion générale **18**

Références **19**

Chapitre 1

Introduction générale

1.1 Contexte et continuité du projet

Dans la première partie du TD, la plateforme E-Bank a été structurée autour de services indépendants : gestion des clients, gestion des comptes, découverte via Netflix Eureka et routage par Spring Cloud Gateway. La deuxième partie ajoute une couche d'interaction en langage naturel sans déplacer la logique métier dans le chatbot. L'agent fait appel à des outils décrivant explicitement les opérations autorisées.

1.2 Objectifs

Les objectifs sont : créer un microservice Spring AI ; exposer les opérations des services métier par MCP ; valider ces outils par MCP Inspector ; connecter le client MCP à un agent ; proposer une API REST ; intégrer Telegram et Discord ; conserver des preuves de tests et analyser les erreurs rencontrées.

1.3 Environnement

Le projet utilise Java 21, Spring Boot 3.5.10, Spring Cloud 2025.0.3, Spring AI 1.1.8, Maven et Visual Studio Code sur macOS. Les services métier utilisent des bases H2 en mémoire, ce qui explique la disparition des données de démonstration après redémarrage.

Chapitre 2

Architecture technique de la partie II

2.1 Organisation des composants

Le port 8761 héberge Eureka ; Customer-Service écoute sur 8056 ; EBank-Service sur 8057 ; EBank-Bot sur 8058 ; Gateway sur 9999. Les routes de passerelle permettent d'atteindre les API REST et les serveurs MCP au moyen des noms de services publiés dans Eureka.

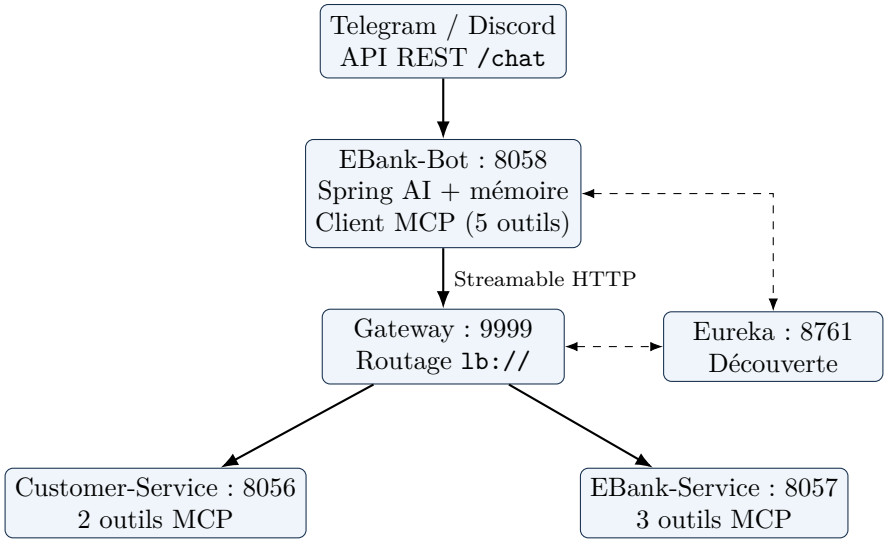


FIGURE 2.1 – Vue logique simplifiée des échanges entre interfaces, agent, passerelle, découverte et outils métier.

2.2 Principes de conception

Les informations bancaires restent gérées par les services métier ; l'agent sélectionne des outils sans accès direct aux bases H2. La séparation limite le couplage entre les canaux de communication et la logique applicative. Les routes MCP `/customer-mcp/mcp` et `/ebank-mcp/mcp` utilisent `StripPrefix=1` avant transmission aux ports métier. La topologie suppose que Gateway et les services sont joignables via les adresses Eureka.

2.3 Enregistrement Eureka

La capture finale du tableau de bord met en évidence quatre instances applicatives *UP* : Customer-Service, EBank-Service, Gateway-Service et EBank-Bot. Une autre capture, plus

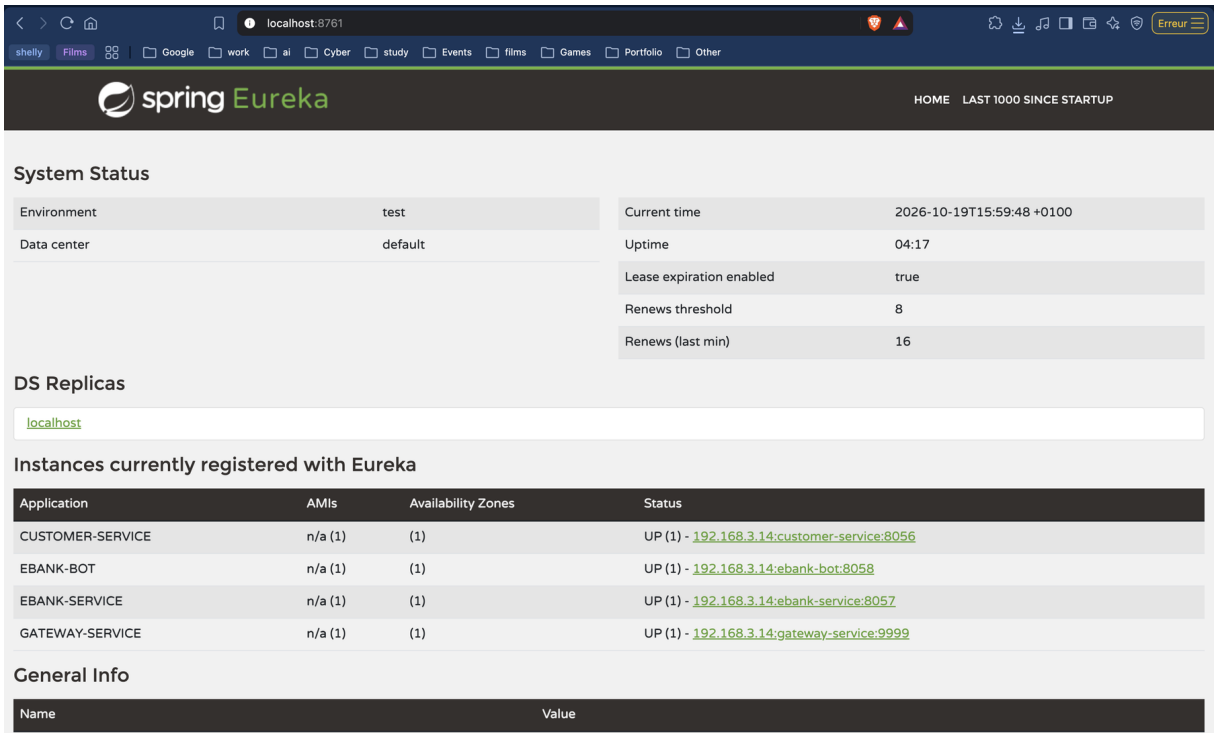


FIGURE 2.2 – Eureka : quatre applications enregistrées avec le statut UP.

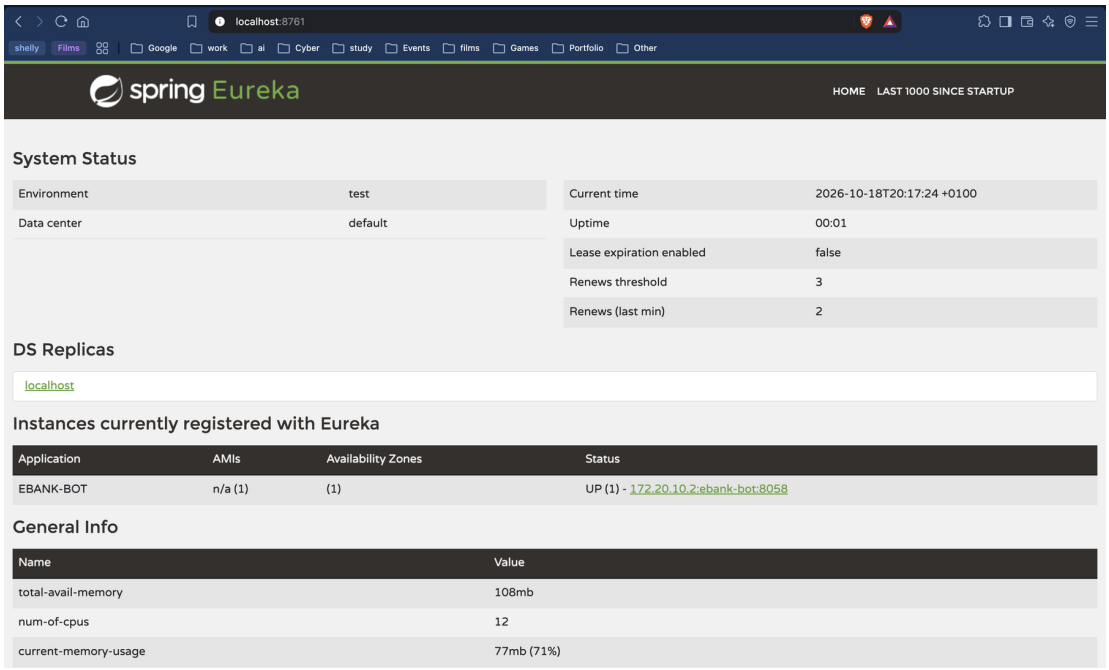


FIGURE 2.3 – Eureka à un stade intermédiaire : EBank-Bot seul apparaît dans la liste.

ancienne, ne montre que l'agent : il s'agit d'un état intermédiaire de démarrage et non de la topologie finale.

Chapitre 3

Création du microservice intelligent

3.1 EBank-Bot et les dépendances

L'application `ebankbot` est un projet Spring Boot sur le port 8058 ; elle comprend le starter Web, le client Eureka, Spring AI OpenAI, le client MCP, Springdoc OpenAPI, et les connecteurs des plateformes de messagerie. Les dépendances sont gouvernées par les BOM Spring Cloud et Spring AI afin d'éviter des combinaisons incohérentes de versions.

3.2 Agent conversationnel

La classe `EbankAiAgent` utilise un `ChatClient` Spring AI, une consigne système en français et une mémoire de conversation `MessageWindowChatMemory` limitée à vingt messages. Chaque appel reçoit un identifiant de conversation pour séparer les échanges. Le fournisseur `SyncMcpToolCallbackProvider` expose les outils distants au client de discussion. Les consignes demandent de ne jamais inventer des clients, des comptes ou des soldes.

Listing 3.1 – Appel conceptuel à l'agent avec mémoire de conversation

```
chatClient.prompt()
    .user(question)
    .advisors(a -> a.param(ChatMemory.CONVERSATION_ID, conversationId))
    .call().content();
```

3.3 API REST et Swagger

L'API `GET /chat` accepte une question et un identifiant de conversation. Springdoc fournit un point d'entrée Swagger pour réaliser un essai manuel. La figure 3.1 affiche un test de présentation de l'agent, mais la réponse HTTP obtenue est **500**. Le formulaire et la route sont ainsi vérifiés, **pas** le succès de la génération par le modèle.

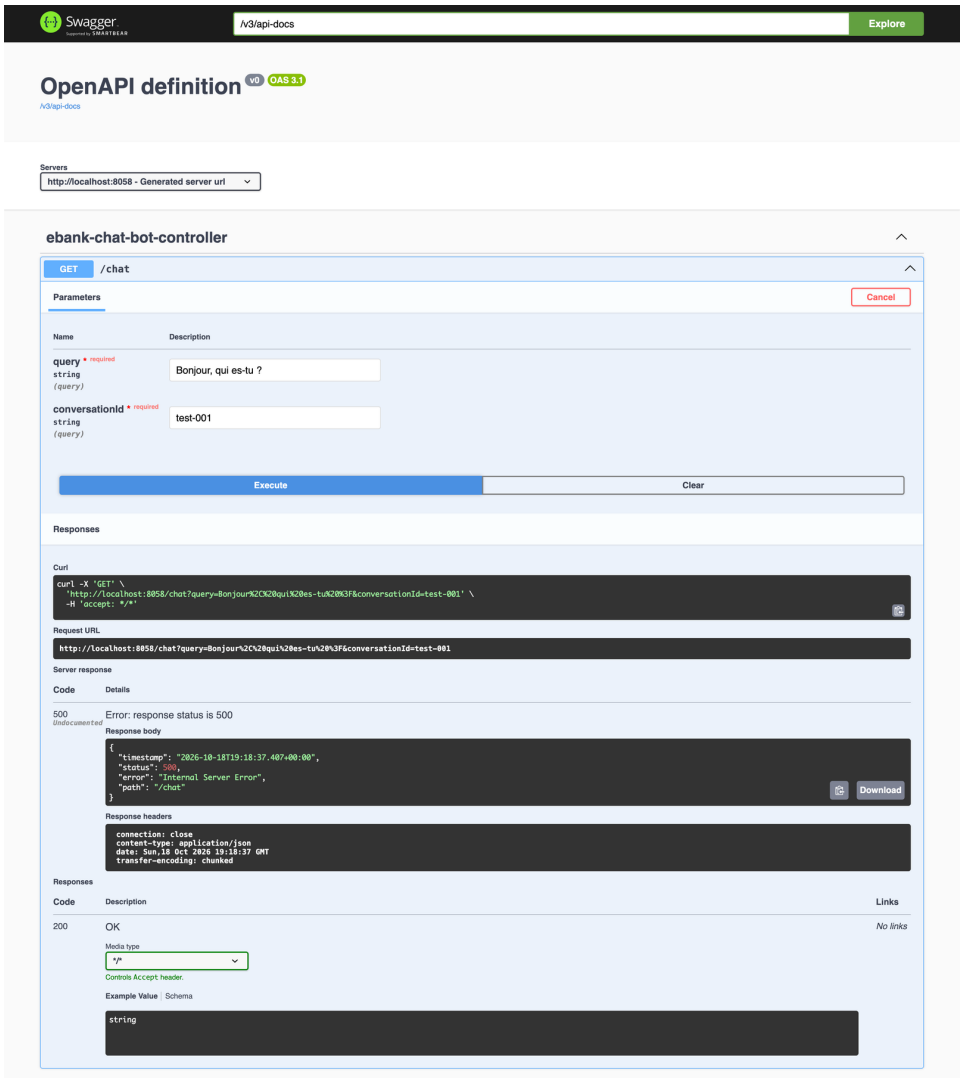


FIGURE 3.1 – Essai Swagger de `GET /chat` : appel effectué, réponse HTTP 500 (limite observée).

Chapitre 4

Publication des outils MCP

4.1 Principes du Model Context Protocol

MCP fournit une convention pour décrire et invoquer des capacités métier à partir d'un client intelligent. Dans cette application, les serveurs MCP utilisent le transport *Streamable HTTP* et des outils annotés en Java. L'approche évite de dupliquer les services REST et explicite le schéma des paramètres.

4.2 Serveur Customer

Deux outils sont publiés : `list_customers`, pour obtenir les clients, et `get_customer_by_id`, pour accéder à un client par son identifiant numérique. Les méthodes annotées délèguent au service métier existant ; elles ne réimplémentent pas l'accès H2.

4.3 Serveur EBank

Trois outils sont publiés : `list_accounts`, `get_account_by_id` et `list_accounts_by_customer`. Ils permettent respectivement de lister les comptes, de rechercher un compte par UUID et de filtrer par identifiant client.

Outil	Description et paramètres
`list_customers`	Liste les clients ; sans paramètre.
`get_customer_by_id`	Récupère un client ; identifiant numérique.
`list_accounts`	Liste les comptes ; sans paramètre.
`get_account_by_id`	Récupère un compte ; UUID.
`list_accounts_by_customer`	Liste les comptes d'un client ; identifiant numérique.

Table 4.1 – Catalogue des cinq outils MCP développés.

4.4 Déclaration dans MCP Inspector

L'outil MCP Inspector a été utilisé afin d'enregistrer les points d'entrée HTTP des deux serveurs puis de consulter leur catalogue. La capture de configuration montre d'abord les deux serveurs dans un état *Disconnected* : elle atteste de leur déclaration, non d'une connexion réussie. Les captures suivantes montrent, séparément, deux et trois outils avec la mention *Connected*.

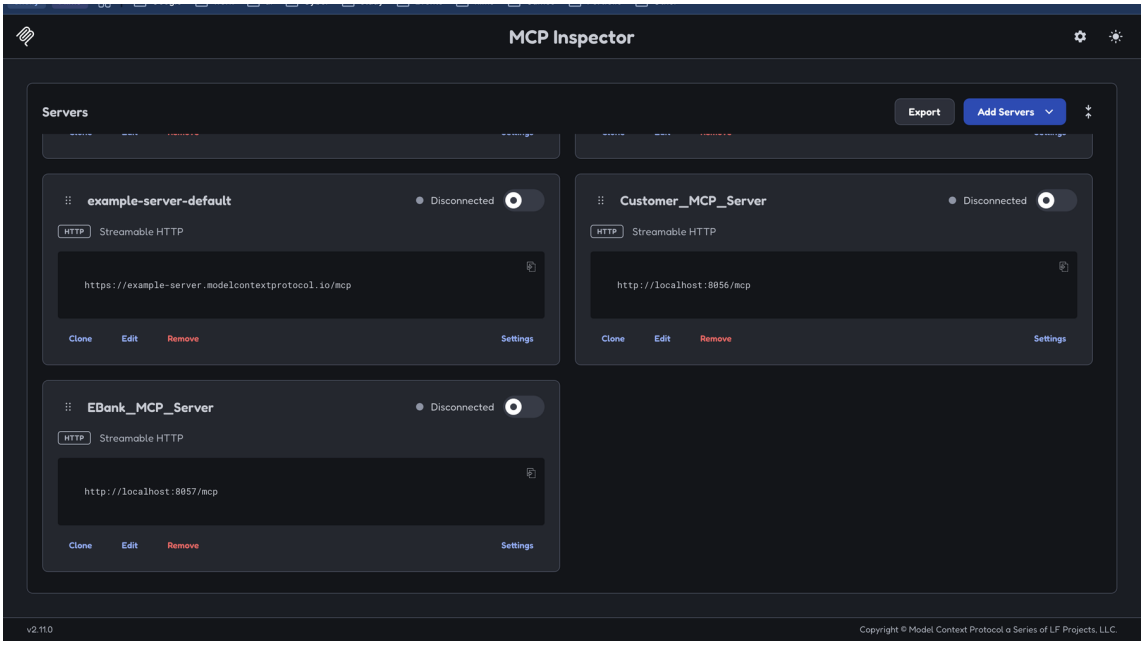


FIGURE 4.1 – Configuration initiale des deux serveurs Streamable HTTP dans MCP Inspector (état non connecté).

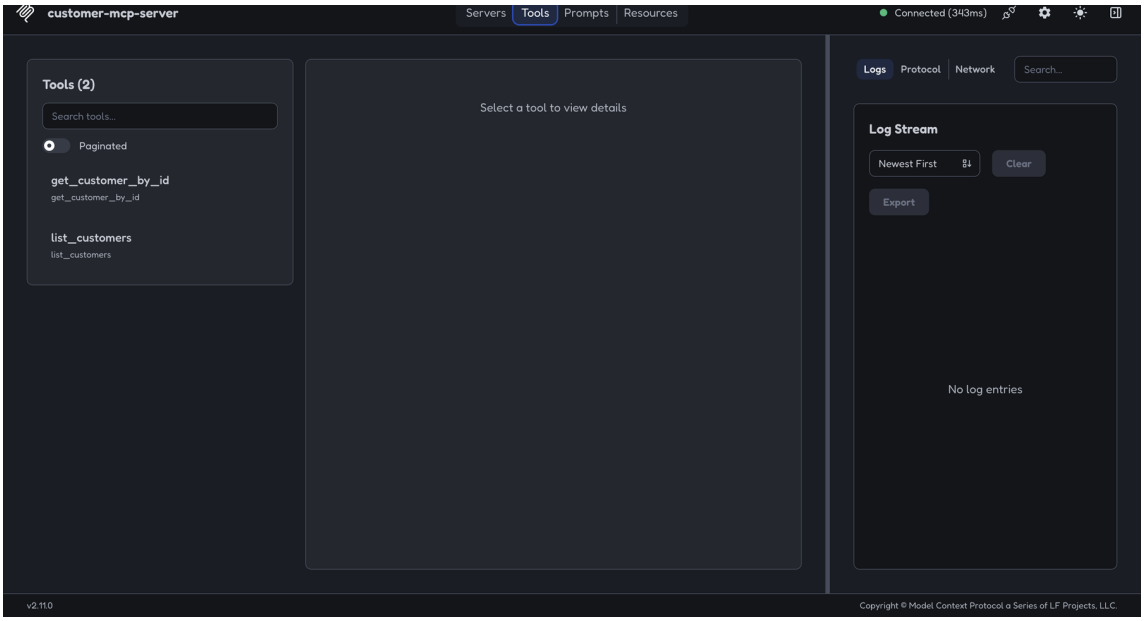


FIGURE 4.2 – MCP Inspector connecté au service clients : deux outils disponibles.

4.5 Exécution des outils

Les appels `list_customers` et `list_accounts` ont permis d'observer des retours de données structurées. Pour le test, un client de démonstration et un compte courant avec un solde de 15 000 ont été insérés en mémoire. Les résultats MCP correspondent aux données de test et portent l'indication `isError: false` dans le terminal.

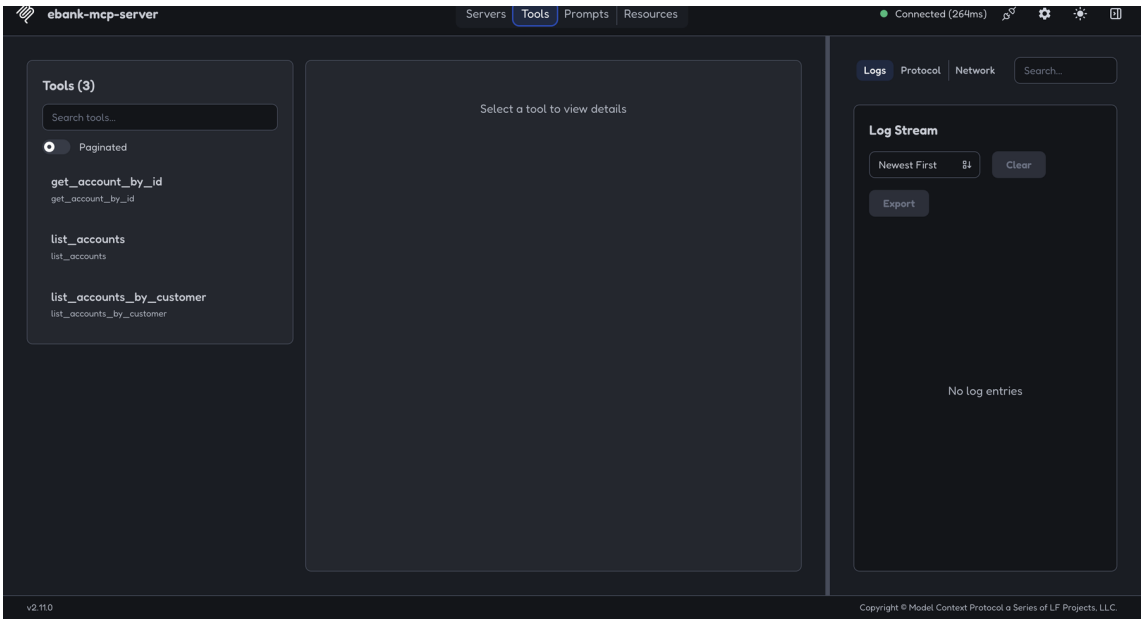


FIGURE 4.3 – MCP Inspector connecté au service bancaire : trois outils disponibles.

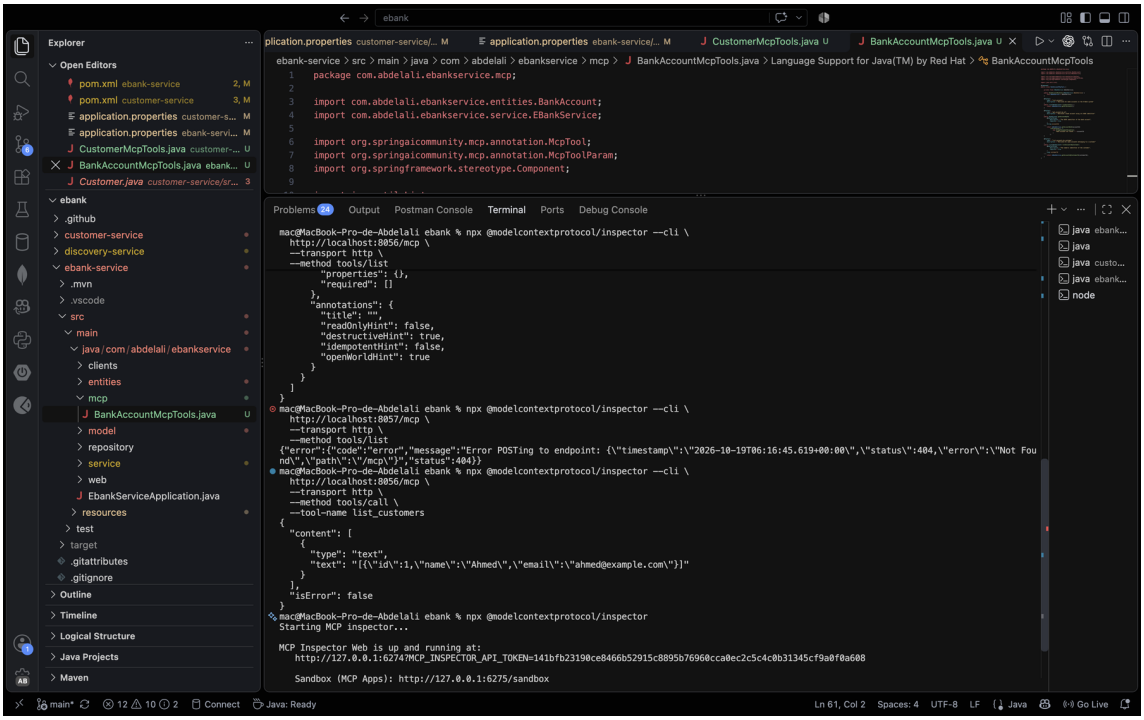


FIGURE 4.4 – Appel de l'outil `list_customers` : réponse contenant un client de démonstration.

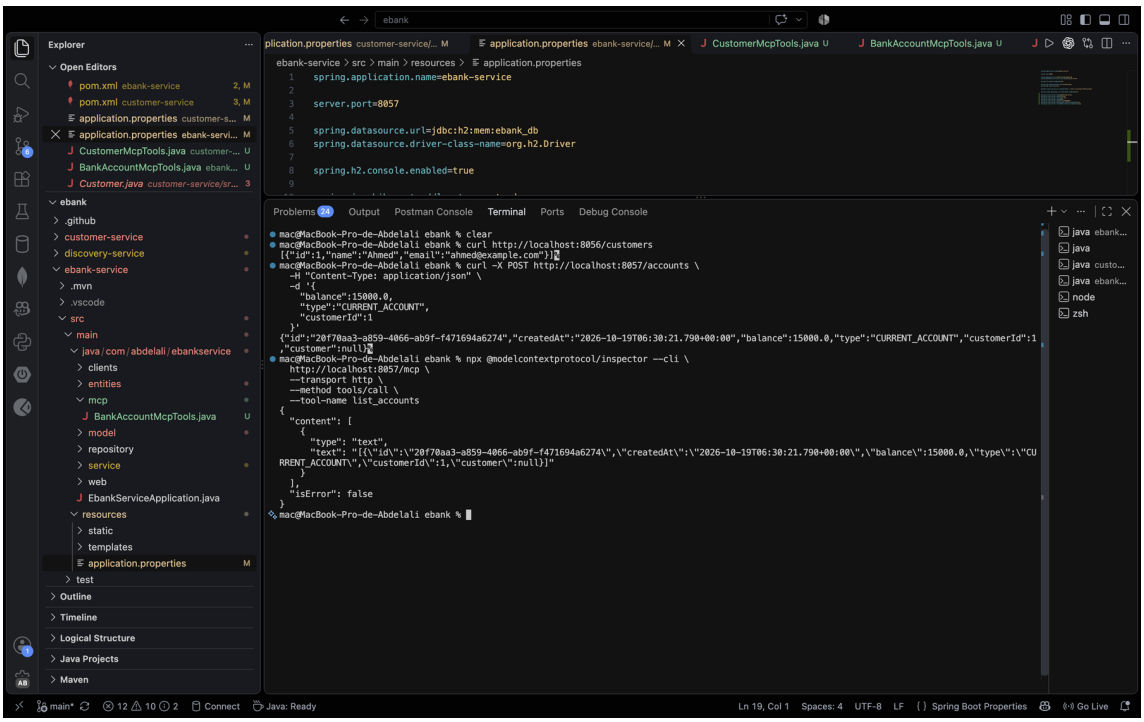


FIGURE 4.5 – Appel de l'outil `list_accounts` : réponse d'un compte courant de démonstration.

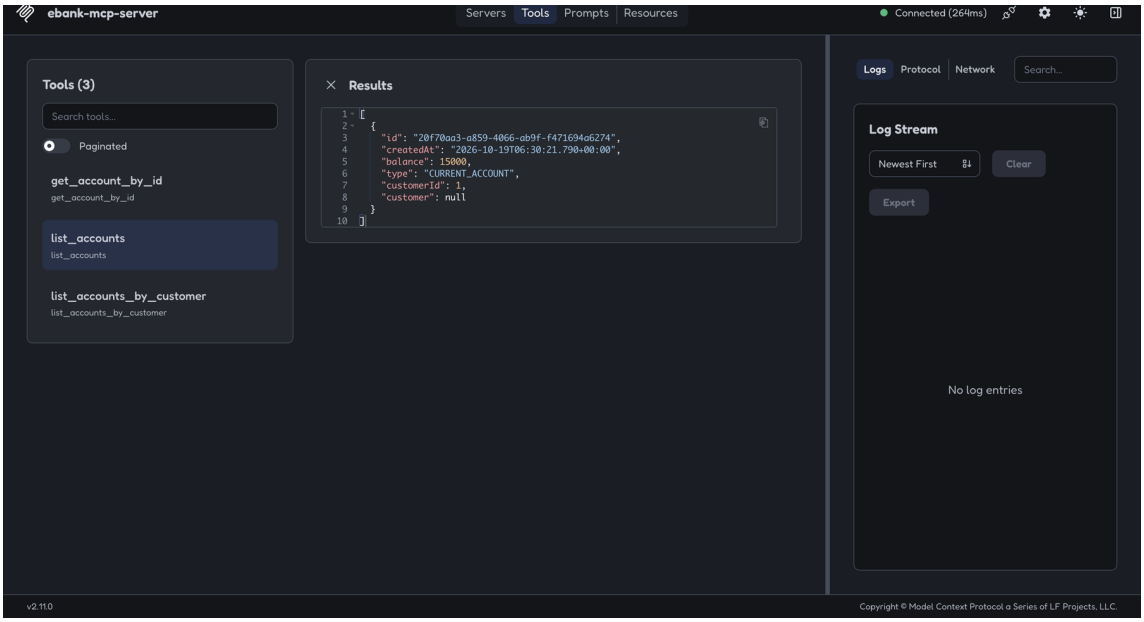


FIGURE 4.6 – Résultat structuré de `list_accounts` visualisé dans MCP Inspector.

Chapitre 5

Intégration du client MCP et de Gateway

5.1 Transport via Gateway

La passerelle route les requêtes métier et MCP à travers la découverte Eureka. Les chemins externes sont `/customer-mcp/mcp` et `/ebank-mcp/mcp`. Chaque route résout le nom de service avec `lb://` et supprime un segment de préfixe pour présenter `/mcp` au serveur.

Listing 5.1 – Exemple de propriétés de routage Spring Cloud Gateway

```
spring.cloud.gateway.server.webflux.routes[2].uri=lb://customer-service
spring.cloud.gateway.server.webflux.routes[2].predicates[0]=Path=/customer-
    mcp/**
spring.cloud.gateway.server.webflux.routes[2].filters[0]=StripPrefix=1
```

5.2 Client MCP Spring AI

Le microservice EBank-Bot initialise deux connexions MCP synchrones. L'agent reçoit les callbacks des outils pour les proposer au modèle et un contrôleur de diagnostic de développement publie les noms disponibles à `/dev/mcp-tools`. Une capture montre le tableau complet des cinq capacités : elle constitue une preuve particulièrement utile de l'assemblage des deux catalogues.

5.3 Difficultés de découverte

Durant l'intégration, certaines annonces Eureka utilisaient des adresses de réseau différentes ou des noms `.local` non résolus par Gateway, entraînant des réponses HTTP 500 et 503. Le diagnostic a consisté à comparer les API directes aux routes de passerelle puis à vérifier la disponibilité des instances Eureka. Dans une exécution où tous les composants restent sur le même poste, une adresse locale cohérente permet de stabiliser leur découverte ; cette solution ne s'applique pas telle quelle au déploiement multi-hôte ou dans Docker.

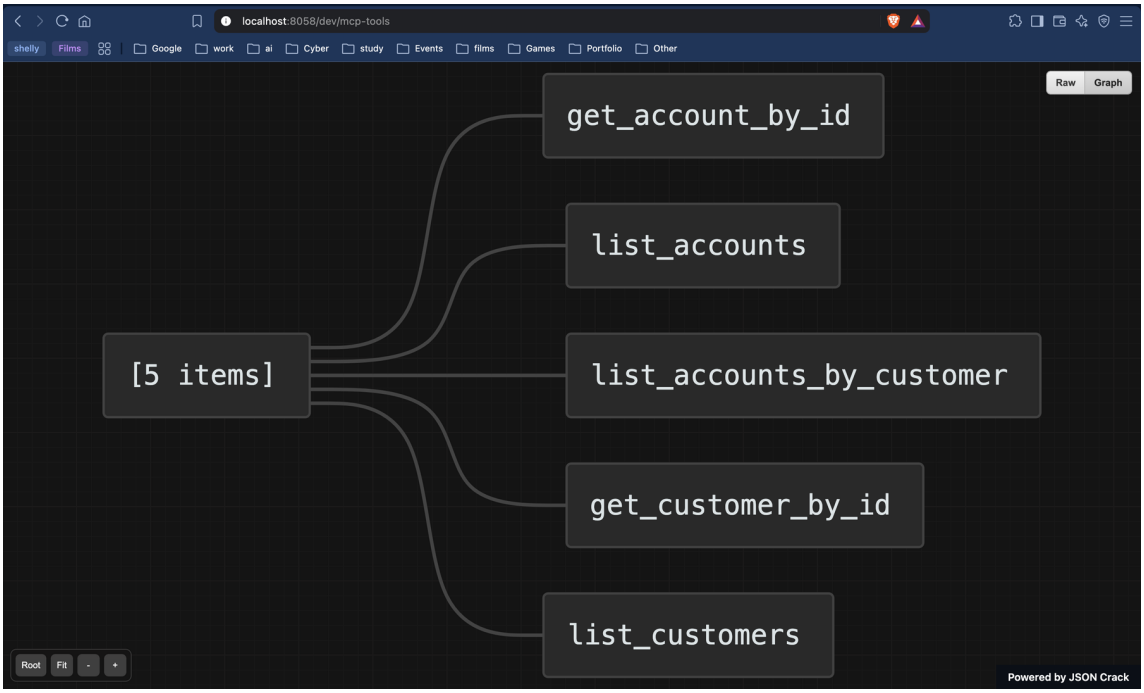


FIGURE 5.1 – Endpoint de diagnostic du chatbot : les cinq outils MCP agrégés.

Chapitre 6

Intégration des interfaces de messagerie

6.1 Telegram

L'interface Telegram s'appuie sur un bot applicatif. Les messages entrants sont réceptionnés par le programme Java, puis transmis à l'agent commun avec un identifiant de conversation propre au canal. Les réponses peuvent être renvoyées à l'utilisateur par l'API Telegram. La capture fournie confirme l'existence du bot et l'envoi de la commande `/start`, **mais ne montre pas de réponse de l'agent**.

6.2 Discord

Une deuxième interface repose sur un bot Discord utilisant JDA. Le canal de test `##banking-bot` a été créé dans un serveur privé ; l'installation prévoit la portée `bot` et les autorisations *View Channels*, *Send Messages* et *Read Message History*. Les messages de salon portant le préfixe `!bank` sont destinés à l'agent. La capture du salon montre la préparation du canal, et celle du portail développeur présente la configuration de l'installation. Aucune interaction question/réponse réussie n'est visible dans ces deux images.

6.3 Mutualisation et limites

Les deux plateformes consomment le même `EbankAiAgent` et, par conséquent, la même surface de cinq outils MCP. Pour une utilisation bancaire réelle, une vérification d'identité et d'autorisation par utilisateur serait indispensable ; l'identifiant Telegram ou Discord ne constitue pas à lui seul une preuve de propriété d'un compte. Les jetons d'intégration doivent rester dans un fichier local exclu de Git.

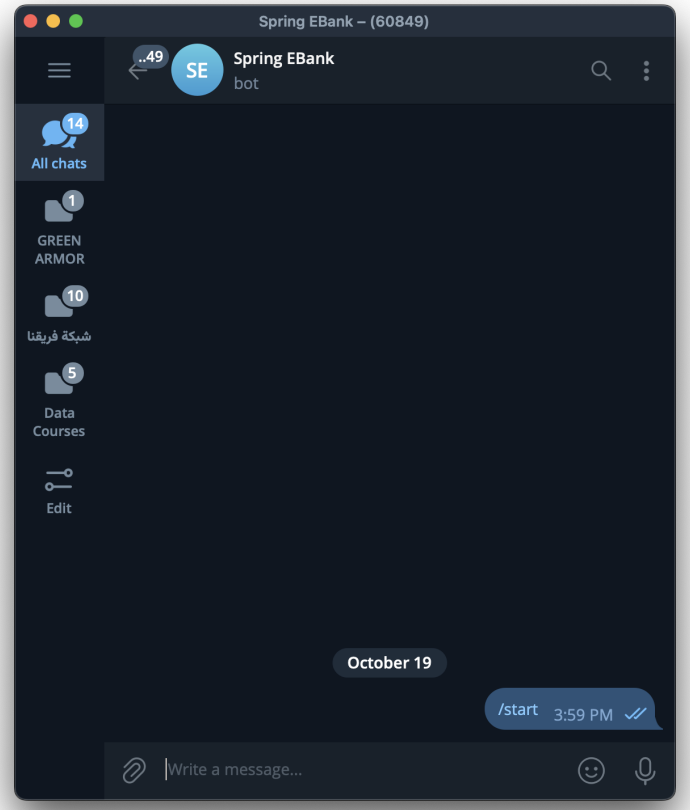


FIGURE 6.1 – Conversation Telegram avec le bot Spring EBank : commande `/start` envoyée.

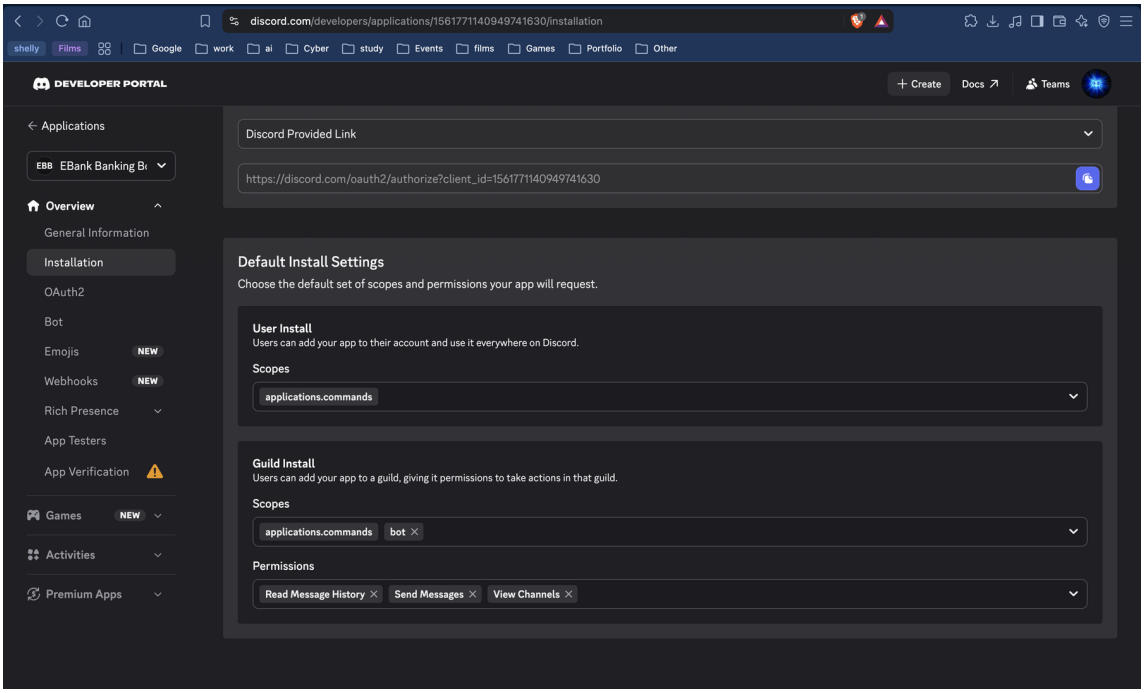


FIGURE 6.2 – Portail Discord Developer : paramètres d'installation, portées et permissions du bot.

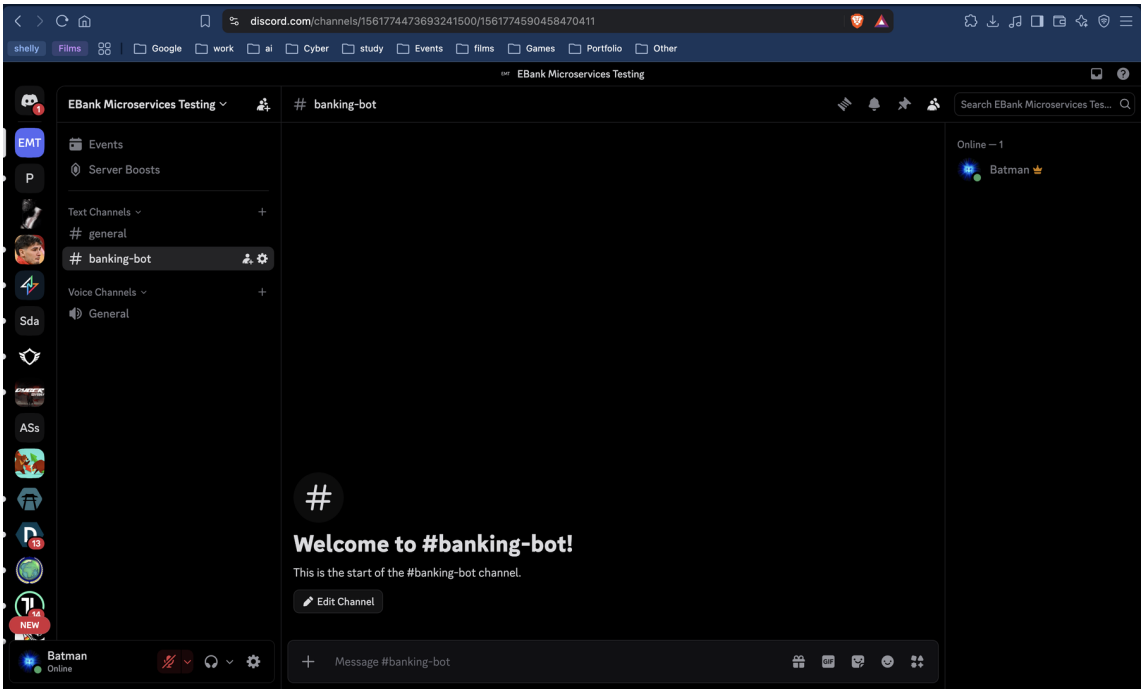


FIGURE 6.3 – Serveur de test Discord et canal `#banking-bot` prêt à recevoir des requêtes.

Chapitre 7

Bilan des validations et perspectives

7.1 Matrice des preuves

Fonction	Statut documenté	Évidence
Enregistrement des services	Observé	Tableau de bord Eureka à quatre instances UP.
Découverte outils clients (2)	Observé	MCP Inspector connecté.
Découverte outils comptes (3)	Observé	MCP Inspector connecté.
Exécution des outils MCP	Observé	Retours de données en terminal et interface Inspector.
Catalogue MCP dans le chatbot (5)	Observé	Endpoint de diagnostic.
Exposition REST de l'agent	Observé, erreur 500	Swagger, appel de `/chat`.
Activation Telegram	Préparation observée	Conversation avec `/start`, sans réponse visible.
Configuration Discord	Préparation observée	Portail développeur et salon de test.
Réponse finale de l'IA dans les messageries	Non démontré par ces captures	Nécessite une preuve de réponse.

Table 7.1 – Synthèse prudente des résultats attestés par les éléments fournis.

7.2 Améliorations envisageables

Les améliorations prioritaires sont une preuve d'interaction complète avec une clé OpenAI valide, la gestion robuste des erreurs par plateforme, la persistance contrôlée des conversations, la sécurisation des outils par autorisation métier et le déploiement dans un environnement reproductible.

Conclusion générale

Cette deuxième partie étend la plateforme E-Bank d'une architecture exclusivement REST vers une approche d'assistance bancaire pilotée par outils. Les opérations métier sont exposées par cinq fonctions MCP indépendantes des interfaces de dialogue ; Spring AI les agrège dans EBank-Bot, tandis que Gateway et Eureka fournissent les mécanismes de routage et de découverte. Les captures confirment la publication et l'exécution d'outils, ainsi que la mise en place des canaux Telegram et Discord. Le test Swagger montrant une erreur 500 et l'absence de réponses de messagerie dans les captures invitent à distinguer la réussite structurelle de la validation complète d'un dialogue IA. Le projet constitue ainsi une base cohérente pour une démonstration finale et pour de futurs travaux de sécurisation.

Références

1. Documentation Spring AI : https://docs.spring.io/spring-ai/reference/
2. Model Context Protocol : https://modelcontextprotocol.io/
3. Spring Cloud Gateway : https://docs.spring.io/spring-cloud-gateway/reference/
4. Spring Cloud Netflix Eureka : https://docs.spring.io/spring-cloud-netflix/reference/
5. API Telegram Bot : https://core.telegram.org/bots/api
6. Discord Developer Documentation : https://discord.com/developers/docs/
7. Repository du projet : https://github.com/AbdelaliSaadali/bdcc-s5-ebank-microservices